CW1 - Digital Forensics for Cybersecurity Students

– subject to changes based on feedback from examiners and questions by students –

Released: Week 7; Due in: Week 10

1 General Information

Note 1: This is a 15-credit module and therefore carries an expected student workload of approximately 150 hours of individual study and assessment activity. Coursework 1 will require approximately 30 hours of individual work (contributing to the module's overall 150-hour workload). Coursework 1 is designed to *assess exactly that type of individual study* aside the material covered in lectures and laboratory sessions. The subjects to research on during this coursework are linked to the lecture in week 5 and specifically `Android` topics. Namely, the main new topics you will encounter in the study towards this coursework are the ones below, accompanied by *example but not definitive* links to study material:

- Deeper acquaintance with the `APK` "anatomy"
- Getting to '.class' files out of an `APK` (i.e., using `dex2jar`)
- What is `Kotlin`, e.g., read `https://kotlinlang.org/`
- Learning about the `Smali` debugger, e.g., read `https://e1mazahy.gitbook.io/droidtomesec/android-pentesting/smali/introduction-to-smali`),
- Inspecting native libs
- Doing checks around the `APK`'s manifest & permissions
- Learning about obfuscation in `Android` , e.g., read`https://www.apriorit.com/dev-blog/obfuscating-code-to-secure-android-apps`

Note 2: How best to solve this coursework? Please read the whole coursework first. Then, read the references mentioned above, or others related to the topics at hand. Then, please try to solve the coursework and, in an iterative process, keep researching about the topics above in the context of the exercises below.

Note 3: This coursework is *individual*. To prove that you yourself did try to solve this coursework you will have to submit printscreens annotated with your username in a particular way described below. Please do not try to counterfeit these printscreens, as it will count as academic misconduct.

Deliverables Checklist

1. One file with all the answers to the below.
2. All scripts (SMS extractor, etc.) and their sample outputs; the sample outputs should be in an appendix of the submission file above.
3. *Evidence Token:* Take and release screenshots mentioned in Note 3 above. Before each screenshot, run `echo $USER-$(date +%s)` and include that one-line token in the same terminal window to prove the screenshot is yours.
4. *Zip/archive* all these and submit ONLY one archive on SurreyLearn.

2 Actual Coursework

Exercise 1: Analysis of an Android File System

Forensics setting: You have been told that malicious actions on the Enterprise Resource Planning (ERP) server originated from the phone of an HR staff member. Determine how the attacker gained access and how ERP credentials leaked.

Download and extract the victim's phone file system archive:

`filesystem.tar.gz`, found on SurreyLearn in the coursework folder.

Tasks (75 pts)

1. Identify the SQLite database that stores SMS/MMS. (5 pts)

   – Provide its absolute path and the exact table name that contains message bodies (Hint: Use `sqlite3` PRAGMA to list schema).

2. Query the database and find any message(s) mentioning login credentials. (5 pts)

   –Hand in: Report the `id` of the SMS, and the exact UTC timestamp(s).

   –Hand in: Show the SQL you executed and the exact output rows.

3. Recover the credentials referenced in the messages. (5 pts)

   –Hand in: Give the absolute path to the file that stores them (if different from SMS DB) and the password as stored at rest.

   –Hand in: Explain whether it is plaintext, encoded, or encrypted.

4. Identify the malicious application: package name, app label, install time, and the sensitive permissions tied to the attack. (10 pts)

   –Hand in: Cite the files you read (e.g., `packages.xml`, `packages.list`, app `AndroidManifest.xml`).

5. Correlate with the Calendar database to explain how/when the app appeared (e.g., social engineering, meeting invite). (10 pts)

   –Hand in: Provide the query used and supporting rows/events.

6. Compute the offset between the phone's app install time and the SMS DB timestamps by comparing at least two independent sources (e.g.[1], `packages.xml` vs. another table). (5 pts)

   Hint: SMS/MMS messages are often a good starting point for gathering information. Locate a SQLite database that stores SMS/MMS messages or information related to them (typically mmssms.db or telephony.db). Use sqlite3 to inspect the schema (.tables and .schema) and identify which table contains message bodies (e.g., sms). Once you have done that, you are compare it with some other table storing install-time information

   Quantify the skew in seconds and discuss implications for a timeline.

[1] "e.g." means "for example", so this is not definite; you need to find AT LEAST two tables, of your choice. There are more than two possibilities.

–Hand in: Report your findings in no more than 15 lines.

7. Write a `bash` or `python` script that: (10 pts)

   i) opens the SMS DB read-only,

   ii) extracts all messages sent within $\pm$ x hours/days of the malicious app's install time (where you can choose a value x, name it and report for that x),

   iii) outputs CSV columns `timestamp_utc, direction, peer, snippet_40chars`, or other columns similar in meaning.

   –Hand in: Include the script in the appendix of your solution submission, and the first 15 lines of its actual output on your dataset.

8. Show a byte-level hex excerpt (20-40 bytes) around where the credential string is stored on disk (Hint: use `xxd` with offset.). (5 pts)

   –Hand in: Show the `xxd` command used.

   –Hand in: Explain encoding (UTF-8? , or UTF-16?) and show the offsets you used.

   Or, if you find credentials in a specific file on the disk, in a form other than a string, report that and details around it.

9. **Minimal Chain-of-Custody Note:** (5 pts)

   Hand in: Draft a short note (max. 1 page) documenting acquisition source (i.e., how you got the `APK`), verification (e.g., you stored it, you hashed it, etc., before you touched it, etc.), and (non-altering) methods you used in the way of forensic science. Hint: Avoid opinions; stick to verifiable steps.

Exercise 2: Android Reverse Engineering (35 points)

Analyse the `APK` provided below, and extract as much information as possible, as per what is asked of you below.

Download the malicious application code:

`base.apk`, found on SurreyLearn in the coursework folder.

In the Tsurugi VM, install and run what you need to `unzip` this `apk`. (5 pts)

1. Determine the language(s) used to build the app from the archive contents (`classes.dex`, `*.so`, presence/absence of `kotlin` metadata, etc.). (5 pts)

   –Hand in: Give the answer to the above and justify with concrete file paths and hexdumps where relevant.

2. List the declared permissions (from `AndroidManifest.xml` or `aapt`). (5 pts)

   –Hand in: Provide the list above, plus exact manifest snippet with line numbers or `XPath` used to extract them.

3. Find the version number of `androidx:core:core` via `META-INF`. (10 pts)

   –Hand in: Provide the version above, plus show the exact file you read and the line(s) where the version appears.

Search only how to decompile `classes.dex` using, e.g., `JADX` (an example way to run in `Tsurugi` is provided below). Use this or something akin you may find online to decompile them. (10 pts)

> Download the jadx-1.5.3.zip from `https://github.com/skylot/jadx/releases/tag/v1.5.3`. Unzip the downloaded file and navigate to `jadx-1.5.3.zip/bin`. Double click on the `jadx-gui` file and once the GUI has loaded, open the `classes.dex` file.
>
> Alternatively, open a terminator and navigate to the `jadx-1.5.3.zip/bin` folder. Once there, run:
>
> ```
> ./jadx -ds output/ /home/user/<path to classes.dex>
> (e.g. ./jadx -ds output/ /home/user/Downloads/classes.dex)
> ```

(Decompilation using the terminator may end with an error line; verify output files exist.)

4. <u>Hand in:</u> What language is the decompiled source? Explain why it may differ from the original (bytecode, obfuscation, inlining).

5. In `output/com/android/updateserver/server`, extract the exact URL/host to which SMS are exfiltrated.

   –<u>Hand in:</u> Provide the method name and the line(s) from source.

6. <u>Hand in:</u> Summarise the role of the second Java file in that directory in 3-4 precise bullet points (method names, IO, exceptions handled). (5 pts)